ECE2277A Project Report

Name: Darsh Patel

Student Number: 251426456

Submission Date: Dec 9th, 2025

Problem Statement

The objective of this project is to design and implement a digital circuit that will be able to identify two given sequences of 4-bits in a serial input stream. When the first sequence of 4-bits (X-sequence: 0001) appears, the system must transition into a mode where the output Z = 1. When the second sequence of 4-bits (Y-sequence: 0011) appears, the machine must return to Z = 0.

To implement this, a Finite State Machine (FSM) is necessary. The FSM has to be made with the help of flip-flops and logical gates, and the assignment instructs students to derive two different circuit implementations using two different state encodings: Gray Code State Assignment, and Reverse Gray Code State Assignment.

After comparing the two designs, the one that implements the problem using the fewest amount of logic components will be chosen as the final circuit design.

Design Method

The design of the sequence detector was approached by breaking down the problem into its basic digital parts and constructing an FSM that would be able to pick out the two required bit patterns. In general, the system involves both combinational and sequential logic. The sequential component includes three D flips-flops storing the machine's current state. These flip-flops, Q2, Q1, and Q0, collectively encode eight such states needed for the detection of "0001" and "0011" sequences. The combinational logic part of the design uses AND, OR, and NOT gates in generating the next-state signals D2, D1, D0 based on the current state and the external input bit X. The circuit also has a reset input Rx, and to bring the system into its initial state under the assumption that this is used before the system starts working. On each rising edge of the clock signal, these values are computed by combinational logic and then loaded into the D flip-flops, making the machine advance according to the state transitions it is designed to follow.

To satisfy the project requirement of generating two different implementations, the entire design process was performed twice. Once using Gray Code State assignment and the other with Reverse Gray Code assignment. Each symbolic state (S0-S7) was assigned to a unique three-bit binary code. These state assignments predefine how the FSM transitions between binary values during state changes, having a major impact on the complexity of the Boolean equations. Having prepared both state assignments, a complete state transition table was developed to illustrate the

next-state values and the output Z for every possible combination of the current state and the input X.

Present State	**Next (x=0)**	**Next (x=1)**	**Output (Z)**
000	001	000	0
001	011	000	0
011	010	000	0
010	010	110	0
110	111	110	1
111	101	110	1
101	101	100	1
100	111	000	1

Figure 1a: State Diagram (Gray Code)

Present State	**Next (x=0)**	**Next (x=1)**	**Output (Z)**
000	100	000	0
100	110	000	0
110	111	000	0
111	111	101	0
101	001	101	1
001	011	101	1
011	011	010	1
010	001	000	1

Figure 1b: State Diagram (Reverse Gray Code)

Minterm	Current	X	Next State
0	000	0	001
1	000	1	000
2	001	0	011
3	001	1	000
6	011	0	010
7	011	1	000
4	010	0	010
5	010	1	110
12	110	0	111
13	110	1	110
14	111	0	101
15	111	1	110
10	101	0	101
11	101	1	100
8	100	0	111
9	100	1	000

Figure 2a: State Table (Gray Code)

Minterm	Current	X	Next State
0	000	0	100
1	000	1	000
8	100	0	110
9	100	1	000
12	110	0	111
13	110	1	000
14	111	0	111
15	111	1	101
10	101	0	001
11	101	1	101
2	001	0	011
3	001	1	101
6	011	0	011
7	011	1	010
4	010	0	001
5	010	1	000

Figure 2b: State Table (Reverse Gray Code)

Such tables were used as the basis for developing truth tables for each D-input.

From the truth tables, Karnaugh maps (K-maps) were generated for D2, D1, and D0 for each state assignment. The K-maps allowed logical simplification by visually grouping adjacent minterms.

	00	01	11	10
00				
01		1		
11	1	1	1	1
10	1		1	1

Figure 3: K-Map for D0

Boolean Expression: Q2Q3'x + Q1x' + Q1Q3

	00	01	11	10
00				1
01	1	1		1
11	1	1	1	
10	1			

Figure 4: K-Map for D1

Boolean Expression: Q1'Q3x' + Q2Q3' + Q1Q3'x' + Q1Q2x

	00	01	11	10
00	1			1
01				
11	1			1
10	1			1

Figure 5: K-Map for D2

Boolean Expression: Q2'x' + Q1x'

The Gray Code and Reverse Gray Code implementations each produced different groupings, thus different Boolean equations. The Gray Code assignment yielded significantly simpler expressions with a fewer number of product terms. For example, the D2 equation simplified to only two terms, whereas the Reverse Gray Code version requires more logic. In addition, the output function Z reduced to a single operation, Z = Q1, since the output depends only on whether the state lies within the higher half of the state diagram.

	00	01	11	10
00				
01				
11	1	1	1	1
10	1	1	1	1

Figure 6: K-Map for Z

Boolean Expression: Q1

Boolean expressions that we obtained, needed to be translated into a Quartus schematic. The D-input equations implemented based on AND and OR gates and their outputs to were connected to corresponding D flip-flops. Feedback from Q outputs back into the combinational logic was provided to maintain correct state advancement, while the reset signal was connected to the asynchronous preset/clear inputs of the flip-flops to enforce the reset state. The block diagram represents a complete hardware implementation of the derived FSM, with clearly separated combinational and sequential sections.

Since the Gray Code implementation required fewer gates overall (43), it was selected as the final design This version minimizes hardware use and greatly reduces propagation delay. This makes it more efficient compared to the Reverse Gray Code implementation.

	Gray Code	**Reverse Gray Code**
# of Gates	12	17
# of inputs	31	47
Total Cost	43 (41 without duplicates)	64 (59 without duplicates)

Figure 7: Cost Comparison

The entire method from establishing state assignments to generating Boolean equations and eventually implementing the hardware provides a well structured and verifiable digital system that reliably detects the two sequences specified in the problem.

Implementation

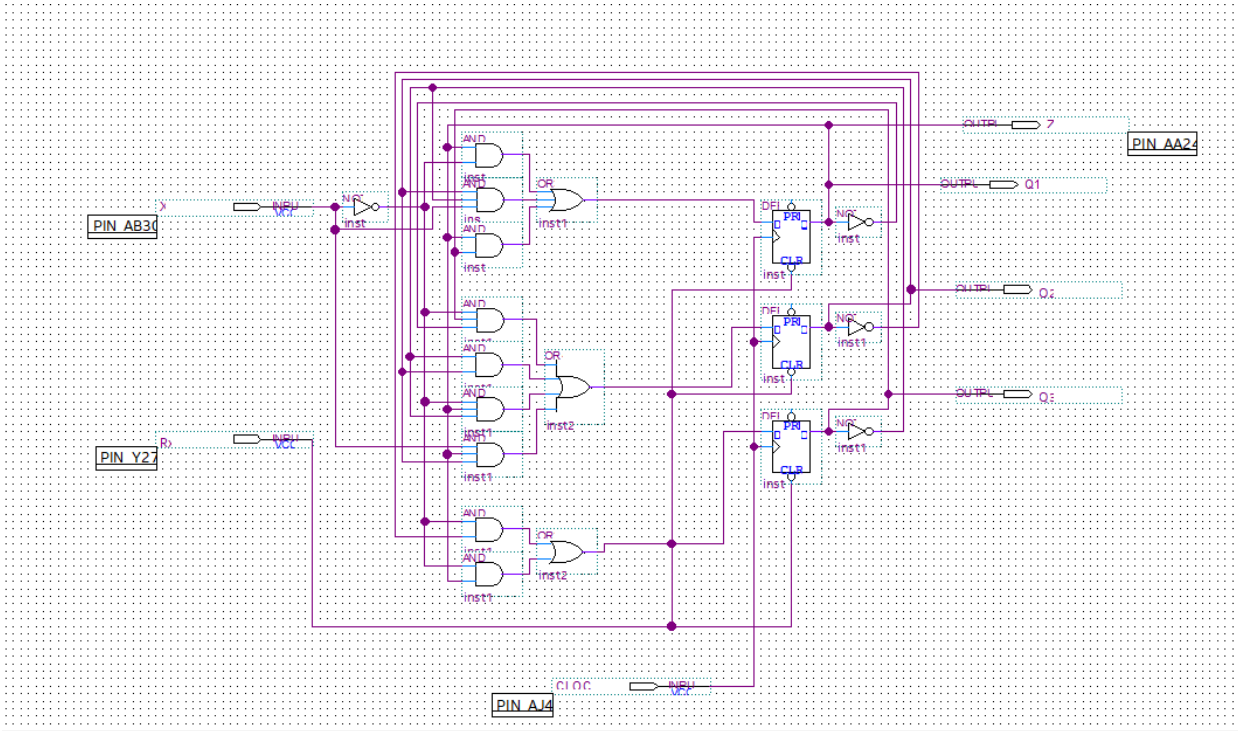

Figure 8: Final FSM Implementation in Quartus.

The three D flip-flops (right side) store Q2-Q0. The left side logic gates compute D2, D1, and D0 based on the current state and the input X. The reset input (Rx) is connected to all three flip-flops asynchronously clear/preset lines to force the rest state. The serial input X comes in through the AB30 pin, and the output state bits and Z are routed to pins on the right to test it.

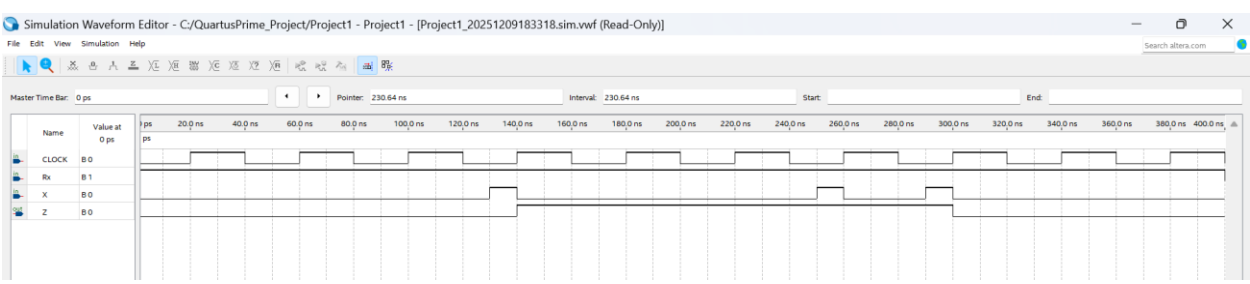

Figure 9: Simulation waveform showing state transitions and correct assertion of Z after detecting 0001 and 0011.

The waveform verifies that the output Z turns on after the 0001 sequence and the output Z turns off after the 0011 sequence, which implies correct operation. Rx is always at high state (1) and clock is counting every 20 ns.

Discussion

The primary design goal was to implement a sequence detector circuit with minimal and efficient design. Two different state assignments, namely Gray Code and Reverse Gray Code, have been tried out since the nature of encoding directly affects the complexity in the resulting logic equations.

The Gray Code produced far simpler Boolean expressions with a reduced overall number of gates. This minimized propagation delay, besides, the wiring complexity being reduced, and the structure of the circuit would be as compact as possible. The structure used state tables, K-maps, Boolean equations, schematic ensured a reliable design rather easy to debug and verify in simulation.

Overall, the implementation of Gray Code was chosen because it had the efficiency, simplicity, and ease of implementation balance in Quartus.